UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática · Ciência da Computação

São Paulo · 2026

Levantamento bibliográfico

Palavras-chave, strings de busca e referências comentadas

ALUNOS Enzo Lopes Campanholo
Felipe Bonatto Zwaizdis Scaquetti
Guilherme Itihara

ORIENTAÇÃO Prof. Me. André Kishimoto

1 Palavras-chave

Instruction set architecture (ISA)
Operational semantics
Specification validation
Theorem proving
Sail, L3, ASL
Separation logic, Iris
CPU emulation
Dynamic binary translation
Meta-tracing
Semantic preservation
Differential testing
Automatic test generation
Test-case reduction
Test oracles
Formal semantics
Executable specifications
Formal verification
Coq, HOL4, Isabelle
Machine code verification
Instruction set simulation
Architecture description languages
Just-in-time compilation
Compiler verification
Functional programming
Fuzzing
Property-based testing
Delta debugging

2 Strings de busca

```
"Sharp SM83" OR LR35902 OR "Game Boy DMG"
```

```
"executable ISA semantics" OR "formal ISA model"
```

```
"architecture description language" OR ADL
```

```
Sail AND ISA
```

```
"formal verification" OR "formal methods"
```

```
("instruction set architecture" OR ISA) AND ("formal specification" OR
semantics) AND (L3 OR Sail OR ASL)
```

```
(Sail OR "authoritative ISA") AND (Coq OR Iris) AND "machine code"
```

```
x86 AND (executable OR formal) AND ISA AND (simulator OR semantics)
```

```
("8-bit" OR microcontroller OR Z80 OR 6502 OR 8080 OR 8051 OR "MCS-51") AND
(Coq OR Isabelle OR HOL4 OR Matita) AND verification
```

```
("CPU emulator" OR "instruction set simulator") AND ("differential testing"
OR "test generation")
```

```
("instruction set simulator") AND ("architecture description" OR generat*)
AND (fast OR JIT)
```

```
(processor OR CPU) AND "differential fuzzing" AND RTL
```

```
("test-case reduction" OR "delta debugging") AND (compiler OR program)
```

```
("Game Boy" OR SM83) AND (formal OR verification OR "differential testing")
```

3 Referências comentadas

Modelagem e validação de ISAs

1 ARMSTRONG, Alasdair et al. ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS. **Proc. ACM Program. Lang.**, New York, NY, USA, v. 3, n. POPL, p. 71:1–71:31, jan. 2019. DOI 10.1145/3290384. Disponível em: https://doi.org/10.1145/3290384.

JUSTIFICATIVA O artigo apresenta a linguagem Sail e modelos semânticos para o comportamento sequencial de arquiteturas como ARMv8-A, RISC-V e CHERI-MIPS. Trata-se de um paralelo claro com a nossa proposta, que é construir um modelo semântico para a arquitetura do SM83.

2 DASGUPTA, Sandeep; PARK, Daejun; KASAMPALIS, Theodoros; ADVE, Vikram S.; ROŞU, Grigore. A complete formal semantics of x86-64 user-level instruction set architecture. *In*: **Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation**. New York, NY, USA: Association for Computing Machinery, 2019. p. 1133–1148. ISBN 9781450367127. DOI 10.1145/3314221.3314601. Disponível em: https://doi.org/10.1145/3314221.3314601.

JUSTIFICATIVA O artigo apresenta uma semântica formal executável das instruções sequenciais de nível de usuário da arquitetura x86-64 Haswell, excluindo as obsoletas. Trata-se de um trabalho análogo ao nosso projeto, embora usemos ferramentas diferentes, com escopo reduzido e hardware mais simples. Pode servir de referência para as estratégias de implementação.

3 GOEL, Shilpi; HUNT, Warren A., Jr.; KAUFMANN, Matt. Engineering a Formal, Executable x86 ISA Simulator for Software Verification. *In*: HINCHEY, Mike; BOWEN, Jonathan P.; OLDEROG, Ernst-Rüdiger (ed.). **Provably Correct Systems**. Cham: Springer International Publishing, 2017. p. 173–209. ISBN 978-3-319-48628-4. DOI 10.1007/978-3-319-48628-4_8. Disponível em: https://doi.org/10.1007/978-3-319-48628-4_8.

JUSTIFICATIVA Em um trabalho sobre o mesmo tema de Dasgupta e colaboradores, os autores constroem um simulador formal executável da ISA x86 em ACL2 e o validam por comparação com um processador físico. É útil como referência de um projeto similar, com escopo e complexidade maiores.

4 WANG, Jiawei; FU, Ming; QIAO, Lei; FENG, Xinyu. Formalizing SPARCv8 instruction set architecture in Coq. **Science of Computer Programming**, *[s. l.]*, v. 187, p. 102371, 2020. ISSN 0167-6423. DOI 10.1016/j.scico.2019.102371. Disponível em: https://www.sciencedirect.com/science/article/pii/S0167642318301163.

JUSTIFICATIVA No artigo, os autores documentam o processo de formalização da arquitetura SPARCv8 em Coq, hoje chamado Rocq. Trata-se de um paralelo claro com o nosso objetivo, que é formalizar a arquitetura do SM83. Os autores, contudo, formalizam a arquitetura diretamente em Coq. Planejamos usar o Sail para gerar as definições necessárias para as provas em Rocq, mas o artigo ainda assim é útil como exemplo de formalização de uma arquitetura.

5 REID, Alastair. Trustworthy specifications of ARM® v8-A and v8-M system level architecture. *In*: **2016 Formal Methods in Computer-Aided Design (FMCAD)**. *[S. l.]:* IEEE, 2016. p. 161–168. DOI 10.1109/FMCAD.2016.7886675.

JUSTIFICATIVA No artigo, Reid, então pesquisador da ARM, descreve um projeto de cinco anos para gerar especificações executáveis das arquiteturas ARMv8-A e ARMv8-M a partir dos mesmos materiais usados na documentação. Essas especificações foram usadas na verificação formal de processadores ARM. O artigo pode nos ajudar a entender como validar especificações executáveis, uma necessidade do nosso projeto.

6 REID, Alastair. Who guards the guards? Formal validation of the Arm v8-M architecture specification. **Proc. ACM Program. Lang.**, New York, NY, USA, v. 1, n. OOPSLA, p. 88:1–88:24, out. 2017. DOI 10.1145/3133912. Disponível em: https://doi.org/10.1145/3133912.

JUSTIFICATIVA O artigo aborda um problema central para o nosso projeto: como saber se as especificações usadas nas provas formais estão corretas? No nosso caso, não dispomos de uma especificação formal oficial do SM83. As estratégias de validação apresentadas podem nos ajudar a avaliar se o nosso modelo corresponde ao comportamento do processador.

Simulação e emulação eficientes

7 BOLZ-TEREICK, Carl Friedrich; PANAYI, Luke; MCKEOGH, Ferdia; SPINK, Tom; BERGER, Martin. Pydrofoil: Accelerating Sail-Based Instruction Set Simulators. *In*: ALDRICH, Jonathan; SILVA, Alexandra (ed.). **39th European Conference on Object-Oriented Programming (ECOOP 2025)**. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. v. 333, p. 3:1–3:31. ISBN 978-3-95977-373-7. ISSN 1868-8969. DOI 10.4230/LIPIcs.ECOOP.2025.3. Disponível em: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2025.3.

JUSTIFICATIVA O artigo apresenta o Pydrofoil, que gera simuladores a partir de modelos em Sail. Nos testes dos autores com RISC-V, a ferramenta obteve uma aceleração superior a 230 vezes em relação ao simulador em C gerado pelo Sail. Como o desempenho é importante para o nosso projeto, podemos considerar essa ferramenta caso o simulador gerado pelo Sail seja lento demais. Esse ganho, porém, não é uma garantia para o SM83.

8 LOCKHART, Derek; ILBEYI, Berkin; BATTEN, Christopher. Pydgin: generating fast instruction set simulators from simple architecture descriptions with meta-tracing JIT compilers. *In*: **2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)**. *[S. l.]:* IEEE, 2015. p. 256–267. DOI 10.1109/ISPASS.2015.7095811.

JUSTIFICATIVA O artigo descreve a criação do Pydgin, uma ferramenta que gera simuladores de instruções com compilação JIT a partir de descrições de arquiteturas. Ainda não sabemos se o desempenho do código gerado pelo Sail será suficiente, mas o artigo pode nos ajudar a identificar melhorias para o simulador ou para o próprio Sail.

9 BELLARD, Fabrice. QEMU, a Fast and Portable Dynamic Translator. *In*: **2005 USENIX Annual Technical Conference (USENIX ATC 05)**. Anaheim, CA: USENIX Association, abr. 2005. Disponível em: https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator.

JUSTIFICATIVA O artigo de Fabrice Bellard descreve a criação do QEMU e sua estratégia de tradução binária dinâmica. Como o propósito final do nosso projeto é a emulação do SM83, é importante entender as estratégias e motivações de um projeto como o QEMU.

Verificação de código de máquina e compiladores

10 SAMMLER, Michael et al. Islaris: verification of machine code against authoritative ISA semantics. *In*: **Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation**. New York, NY, USA: Association for Computing Machinery, 2022. p. 825–840. ISBN 9781450392655. DOI 10.1145/3519939.3523434. Disponível em: https://doi.org/10.1145/3519939.3523434.

JUSTIFICATIVA O artigo descreve a ferramenta Islaris, que usa modelos de ISA escritos em Sail para verificar código de máquina. Ela combina a execução simbólica do Isla com provas em Iris e Coq, hoje chamado Rocq. Talvez não tenha aplicação direta ao nosso projeto, mas pode nos ajudar a entender como usar modelos de ISA em provas formais.

11 LEROY, Xavier. Formal verification of a realistic compiler. **Commun. ACM**, New York, NY, USA, v. 52, n. 7, p. 107–115, jul. 2009a. ISSN 0001-0782. DOI 10.1145/1538788.1538814. Disponível em: https://doi.org/10.1145/1538788.1538814.

JUSTIFICATIVA O autor do artigo, Xavier Leroy, é um dos desenvolvedores do CompCert, um compilador de C formalmente verificado. Estamos explorando a possibilidade de usar o CompCert no nosso projeto, mas, mais do que isso, o CompCert é um exemplo de software formalmente verificado que usa Coq, hoje chamado Rocq, uma tecnologia que também usaremos no projeto.

12 LEROY, Xavier. A Formally Verified Compiler Back-end. **Journal of Automated Reasoning**, *[s. l.]*, v. 43, n. 4, p. 363–446, 1 dez. 2009b. ISSN 1573-0670. DOI 10.1007/s10817-009-9155-4. Disponível em: https://doi.org/10.1007/s10817-009-9155-4.

JUSTIFICATIVA O artigo complementa o anterior. Leroy se aprofunda nos detalhes do back-end do compilador e de suas provas formais. Será útil para entendermos as limitações do CompCert e evitarmos superestimar suas capacidades.

Geração de testes e testes baseados em propriedades

13 MARTIGNONI, Lorenzo; PALEARI, Roberto; ROGLIA, Giampaolo Fresi; BRUSCHI, Danilo. Testing CPU emulators. *In*: **Proceedings of the Eighteenth International Symposium on Software Testing and Analysis**. New York, NY, USA: Association for Computing Machinery, 2009. p. 261–272. ISBN 9781605583389. DOI 10.1145/1572272.1572303. Disponível em: https://doi.org/10.1145/1572272.1572303.

JUSTIFICATIVA O artigo apresenta uma estratégia de testes para emuladores de CPU baseada em fuzzing e na comparação com processadores físicos. Um dos objetivos do projeto é comparar nosso modelo com outros emuladores e, possivelmente, núcleos implementados em FPGA. O artigo servirá de base para a montagem dos nossos testes automatizados.

14 JIANG, Muhui et al. EXAMINER: automatically locating inconsistent instructions between real devices and CPU emulators for ARM. *In*: **Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems**. New York, NY, USA: Association for Computing Machinery, 2022. p. 846–858. ISBN 9781450392051. DOI 10.1145/3503222.3507736. Disponível em: https://doi.org/10.1145/3503222.3507736.

JUSTIFICATIVA O artigo descreve uma ferramenta que encontra diferenças no comportamento de instruções entre emuladores ARM e dispositivos reais. Ela gera testes a partir da execução simbólica da especificação em ASL. Ainda não sabemos se teremos um dispositivo real para testes, mas, caso consigamos, podemos adaptar parte dessa estratégia ao nosso projeto.

15 CAMPBELL, Brian; STARK, Ian. Extracting behaviour from an executable instruction set model. *In*: **2016 Formal Methods in Computer-Aided Design (FMCAD)**. *[S. l.]:* IEEE, 2016. p. 33–40. DOI 10.1109/FMCAD.2016.7886658.

JUSTIFICATIVA No artigo, os autores discutem como modelos de conjuntos de instruções escritos como funções executáveis podem dificultar a análise do comportamento de cada instrução. Eles apresentam um método que combina avaliação e execução simbólicas em HOL4 para extrair regras de comportamento e gerar testes. Isso pode nos auxiliar na construção dos testes automatizados para o nosso modelo.

16 PARASKEVOPOULOU, Zoe; HRIŢCU, Cătălin; DÉNÈS, Maxime; LAMPROPOULOS, Leonidas; PIERCE, Benjamin C. Foundational Property-Based Testing. *In*: URBAN, Christian; ZHANG, Xingyuan (ed.). **Interactive Theorem Proving**. Cham: Springer International Publishing, 2015. p. 325–343. ISBN 978-3-319-22102-1. DOI 10.1007/978-3-319-22102-1_22. Disponível em: https://doi.org/10.1007/978-3-319-22102-1_22.

JUSTIFICATIVA No artigo, os autores apresentam uma metodologia para verificar formalmente testes baseados em propriedades, implementada no QuickChick para Coq. Embora não trate de testes de CPU, o trabalho pode nos ajudar a testar propriedades do modelo em Rocq antes de tentar prová-las e a verificar se os testes avaliam as propriedades pretendidas. Os testes complementam as provas, mas não as substituem.

Redução de casos de teste

17 REGEHR, John; CHEN, Yang; CUOQ, Pascal; EIDE, Eric; ELLISON, Chucky; YANG, Xuejun. Test-case reduction for C compiler bugs. *In*: **Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation**. New York, NY, USA: Association for Computing Machinery, 2012. p. 335–346. ISBN 9781450312059. DOI 10.1145/2254064.2254104. Disponível em: https://doi.org/10.1145/2254064.2254104.

JUSTIFICATIVA Caso um erro seja detectado durante os testes automatizados, como podemos simplificar o caso que o provoca? Neste artigo, os autores apresentam estratégias para reduzir programas em C que expõem erros de compiladores, preservando a falha. Essas estratégias podem inspirar a redução de sequências de instruções nos nossos testes, o que pode acelerar o diagnóstico dos erros.

18 ZELLER, Andreas; HILDEBRANDT, Ralf. Simplifying and isolating failure-inducing input. **IEEE Transactions on Software Engineering**, *[s. l.]*, v. 28, n. 2, p. 183–200, 2002. DOI 10.1109/32.988498.

JUSTIFICATIVA Artigo relacionado ao de Regehr e colaboradores que apresenta o *delta debugging*, uma técnica para simplificar entradas que provocam falhas e isolar diferenças entre casos que passam e falham. Pode nos ajudar a diagnosticar as falhas em nossos testes mais rapidamente.

Oráculos de teste

19 BARR, Earl T.; HARMAN, Mark; MCMINN, Phil; SHAHBAZ, Muzammil; YOO, Shin. The Oracle Problem in Software Testing: A Survey. **IEEE Transactions on Software Engineering**, *[s. l.]*, v. 41, n. 5, p. 507–525, 2015. DOI 10.1109/TSE.2014.2372785.

JUSTIFICATIVA O artigo examina o problema do oráculo de testes: como decidir se um resultado está correto? Uma diferença entre dois resultados não indica qual dos dois está correto. Caso encontremos diferenças em relação a outros emuladores ou núcleos, não podemos concluir que o nosso resultado é o correto apenas por vir de um modelo formal. O artigo também explora abordagens para automatizar essa avaliação, que podem ser úteis nos nossos testes.

Fundamentos da programação funcional

20 BACKUS, John. Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. **Commun. ACM**, New York, NY, USA, v. 21, n. 8, p. 613–641, ago. 1978. ISSN 0001-0782. DOI 10.1145/359576.359579. Disponível em: https://doi.org/10.1145/359576.359579.

JUSTIFICATIVA Artigo clássico de Backus sobre como o computador de von Neumann influencia as linguagens de programação convencionais e limita suas possibilidades de composição e raciocínio matemático. Backus argumenta que o "estilo von Neumann" carece de propriedades matemáticas úteis para o raciocínio sobre programas e propõe uma alternativa funcional. O artigo é útil para fundamentar a importância do raciocínio matemático sobre programas na nossa proposta de verificação formal.